
Project 3 Report: C-Minus Semantic Analyzer

1. Introduction

The objective of this project was to implement a **semantic analyzer** for the C-Minus language. Following the syntactic analysis phase, this component is responsible for validating the logical consistency of the program. Key responsibilities include managing nested scopes via a symbol table, enforcing type rules (type checking), and detecting semantic errors such as undeclared variables, invalid array indexing, or function argument mismatches.

The analyzer was built by extending the infrastructure from the previous parser project, modifying the symbol table to support scoping, and implementing a two-pass traversal strategy for analysis.

2. Environment and Compilation Method

2.1. Development Environment

The project was developed and compiled in a Linux environment using the provided Docker container (cs-compiler-hw:1.0). The key tools used were:

- **C Compiler:** gcc
- **Build System:** make
- **Base Code:** The scanner (flex) and parser (bison) from Project 2 were integrated.

2.2. Compilation Process

The project is compiled using the provided Makefile. The build process generates the `cminus_parser` executable, which now includes the semantic analysis logic.

1. `make clean`: Removes previous build artifacts.
2. `make cminus_semantic`: Compiles and links all modules.

```
root@6263971350af:/work# make cminus_semantic
yacc -d -v cminus.y
gcc -W -Wall -g -c main.c
gcc -W -Wall -g -c util.c
flex cminus.l
gcc -W -Wall -g -c lex.yy.c
lex.yy.c:1340:16: warning: 'input' defined but not used [-Wunused-function]
1340 |     static int input (void)
      |                  ^~~~~~
lex.yy.c:1297:17: warning: 'yyunput' defined but not used [-Wunused-function]
1297 |     static void yyunput (int c, char * yy_bp )
      |                  ^~~~~~
gcc -W -Wall -g -c y.tab.c
gcc -W -Wall -g -c symtab.c
gcc -W -Wall -g -c analyze.c
gcc -W -Wall -g main.o util.o lex.yy.o y.tab.o symtab.o analyze.o -o cminus_semantic -lfl
root@6263971350af:/work#
```

3. Implementation Details

The implementation focused on managing the Symbol Table (symtab.c) and the Semantic Analysis logic (analyze.c).

3.1. Global Data Structures (globals.h)

The globals.h file was modified to support semantic features :

- **Node Kinds:** Expanded NodeKind to include DeclK and its subtypes (VarK, FunK, ParamK).
- **Expression Types:** Defined ExpType (Void, Integer, Boolean, Undetermined).
- **Semantic Attributes:** Updated TreeNode to include fields such as type and isArray.

3.2. Scoped Symbol Table (symtab.c)

The symtab.c file was rewritten to replace the original global hash table with a **stack-based architecture** to handle nested scopes .

- **Scope Stack:** A ScopeList structure was implemented. sc_push and sc_pop are used when entering and exiting functions or compound statements.
- **Hierarchical Lookup:** st_lookup searches for symbols starting from the current scope up to the global scope. st_lookup_top searches only the current scope to detect redefinitions.
- **Extended Information:** The BucketList structure now stores a pointer to the AST node (node), allowing for parameter verification later.

3.3. Pass 1: Symbol Table Construction (analyze.c)

The buildSymtab function performs a pre-order traversal of the AST:

1. **Built-in Functions:** Manually inserts input() and output() into the global scope using addBuiltinFunctions() before traversal begins .
2. **Scope Management:** Detects FunK and CompoundK nodes to manage the scope stack.
3. **Insertion and Verification:** Inserts declarations into the table and verifies the usage of variables and functions. If a symbol does not exist, an error is reported, and it is marked as Undetermined to prevent cascading errors.

3.4. Pass 2: Type Checking (analyze.c)

The typeCheck function performs a post-order traversal to validate type rules :

- **Operations:** OpK requires integer operands (no arrays or void).
- **Assignments:** Verifies type compatibility between the left-hand side and right-hand side.

- **Conditionals:** if and while statements require integer conditions.
 - **Calls:** Uses checkCall to validate that arguments match the defined function parameters in both number and type.
-

4. Complications and Solutions

4.1. Complication: Nested Scope Management

- **Problem:** The original implementation used a single global hash table. C-Minus allows variables with the same name in different scopes (shadowing).
- **Solution:** A linked list of hash tables (ScopeList) was implemented. Each node represents a scope and points to its "parent." This allows multiple variables named x to exist at different levels without collision.

4.2. Complication: Parameter Verification

- **Problem:** The original symbol table did not store parameter information, making it impossible to verify function calls (CallK).
- **Solution:** A struct treeNode * node pointer was added to the BucketListRec structure. When declaring a function, its AST node is saved. In checkCall, this node is retrieved, and its children are traversed to compare actual argument types against formal parameters.

4.3. Complication: Array vs. Scalar Distinction

- **Problem:** Expressions like $x + 1$ are invalid if x is an array, but valid if x is an integer or an indexed element $x[0]$.
 - **Solution:** An isArray flag was added. In analyze.c, the IdK logic determines if an identifier is being used as a full array or an indexed element. If it has an index child, it evaluates to Integer; otherwise, it retains its array property, allowing detection of illegal operations.
-

5. Results and Verification

The analyzer was successfully validated using four specific test cases covering both correct functionality and error detection.

5.1. Test 1: Basic Functionality & Built-ins (test_1.cm)

This test (Euclid's Algorithm) verifies the correct insertion of functions, parameters, and local variables, as well as the usage of the predefined input and output functions .

```

root@eaf5006f159c:/work# ./cminus_semantic test_1.cm
C-MINUS COMPILATION: test_1.cm
Building Symbol Table...

< Symbol Table >
Symbol Name  Symbol Kind  Symbol Type  Scope Name  Location  Line Numbers
-----
main          Function    void         global      3         11
input         Function    int          global      0         0 14 14
output        Function    void         global      1         0 15
gcd           Function    int          global      2         4 7 15
value         Variable    int          output      0         0
u             Variable    int          gcd         0         4 6 7 7
v             Variable    int          gcd         1         4 6 7 7
x             Variable    int          main        0         13 14 15
y             Variable    int          main        1         13 14 15

< Functions >
Function Name  Return Type  Parameter Name  Parameter Type
-----
main           void
input          int
output         void
-             -
gcd            int
-             -
-             -
-             -
-             -
-             -

< Global Symbols >
Symbol Name  Symbol Kind  Symbol Type
-----
main          Function    void
input         Function    int
output        Function    void
gcd           Function    int

< Scopes >
Scope Name  Nested Level  Symbol Name  Symbol Type
-----
output      1            value       int
gcd         1            u           int
gcd         1            v           int
main        1            x           int
main        1            y           int

Checking Types...
Type Checking Finished
root@eaf5006f159c:/work#

```

5.2. Test 2: Scopes and Arrays (test_2.cm)

Verifies the handling of arrays and nested scopes. The variable `i` and array `x` are declared inside `main`.

```

< Scopes >
Scope Name  Nested Level  Symbol Name  Symbol Type
-----
output      1            value       int
main        1            i           int
main        1            x           int[]

```

5.3. Test 3: Function Argument Error (test_3.cm)

This test defines `int x(int y)` but attempts to call it with three arguments: `x(a, b, c)`.

```

Checking Types...
Error: Invalid function call at line 12 (name : "x")

```

5.4. Test 4: Array Indexing Error (test_4.cm)

This test attempts to use `output(5)` (which has a `void` return type) as an index for an array: `x[output(5)]`.

```

Checking Types...
Error: Invalid array indexing at line 4 (name : "x"). indices should be integer

```

6. Conclusion

The semantic analyzer for C-Minus has been fully implemented, meeting all requirements for scope management and type verification. The transition from a flat symbol table to a stack-based one correctly handles C-style scoping. The tests performed demonstrate that the compiler correctly identifies valid code and reports specific semantic errors such as argument discrepancies and invalid type usage, providing a solid foundation for code generation.